

Real-Time Fake PTP Detection & Operational Escalation Engine

Multimodal Speech & NLP Feature Synthesis, Account-Grouped Cross-Validation, and Hold-vs-No-Hold Optimization

Problem Statement 1 (CreditNirvana) – OpenIIT Data Science Competition

Dataset: 3,695 Uncensored PTP Promises across 3,696 Logs | Single-Row Model Latency: 0.663 ms | E2E Latency: 2.185 ms

Executive Summary & Primary Empirical Findings:

In consumer credit debt collections, borrowers frequently issue verbal Promises-to-Pay (PTP) during high-pressure collector calls merely as an escape mechanism. When collection operations place an unconditional hold on these accounts, collection attempts cease, losing critical recovery momentum while 72.45% of promises are ultimately broken.

Key Findings Established in this Investigation:

- **Multimodal Value-Add (Leading Result):** Conversational speech kinetics (+0.0363) and out-of-fold Hinglish character n-gram NLP (+0.0603) deliver a combined **+0.0966 AUC lift** over pre-capture payment history. Combined with PTP terms and salary proximity (+0.0706), total multimodal feature synthesis lifts Test ROC-AUC from **0.6104** (core financials alone) to **0.7753**, delivering an audited paired cluster-bootstrap lift of **+0.1649** (95% CI: [+0.110, +0.227]).
- **Lead-Time Dynamics & Window Duration Artifact:** Contrary to naive intuition, kept rates rise overall with a sharp non-linear jump after 10 days (18.6% -> 30.3% -> 23.3% -> 25.9% -> 50.9% -> 51.4%). We demonstrate that this increase is substantially driven by an operational evaluation window duration artifact: under standard evaluation [t_capture, promised_date + 2d], a 30-day promise affords a 32-day observation window vs 4 days for a 2-day promise. Evaluated on a fixed 3-day window centered on the promised date, the 16–30d kept rate drops from 51.43% to 28.78%.
- **Operational Payoff Optimization:** Framing the operational decision as **Escalate (No-Hold)** when $P(\text{Broken}) \geq t$ under stated operational unit friction cost assumptions ($a = ₹250$ unnecessary customer touch friction, $b = ₹200$ unrecovered default loss avoided), validation tuning identifies an interior optimal threshold $t = 0.57$. On unseen test accounts ($N = 563$), this yields **₹53,800 net savings**, achieving an audited **+₹11,400 lift over Escalate-All** (95% Cluster-Bootstrap CI: [+₹6,700, +₹15,800]), capturing 369 of 407 broken promises (90.7% recall) while safely holding 76 genuine payers (48.7%).
- **Model Parity & Inference Latency:** Across 6 model configurations spanning 4 foundational architectures (Linear, Tree, Neural, Gradient Boosted) evaluated via 5-fold GroupKFold by account ($N = 3,132$), test performances tightly cluster between 0.773 and 0.782. Given a test cluster-bootstrap SE ≈ 0.024 (paired CatBoost vs LogReg difference: +0.0069, 95% CI: [−0.007, +0.022]), base models are statistically indistinguishable. The Platt-Calibrated Blend is deployed for ensembling robustness. Single-row CatBoost inference executes in **0.663 ms** (LogReg: **0.147 ms**; end-to-end extraction + scoring: **2.185 ms**), well within assumed telephony SLAs (< 5.0 ms).

Contents

1 Problem Formulation & Data Architecture	3
1.1 1.1 Business Context & The PTP Hold Dilemma	3
1.2 1.2 Deterministic FIFO Attribution & Censoring Retraction	3

1.3	1.3 Snapshot Leakage Audit on <code>accounts.csv</code>	3
2	Empirical Lead-Time Dynamics & Behavioral Mechanisms	3
2.1	2.1 The Lead-Time Distribution & Evaluation Window Artifact	3
2.2	2.2 Domain Explanation: Non-Monotonic Jump & Window Artifact	4
3	Feature Engineering & Step-by-Step Ablation Study	4
3.1	3.1 Multimodal Feature Architecture	4
3.2	3.2 Step-by-Step Feature Ablation with Paired Cluster-Bootstrap Cls	4
4	Ghost PTP Audit & Latency Profiling	5
4.1	4.1 Ghost PTP Investigation (Under 15s vs Under 20s)	5
4.2	4.2 Measured Inference Latency & Telephone SLA	6
5	Model Benchmark & Architecture Selection	6
5.1	5.1 Selection Protocol & 5-Fold GroupKFold Cross-Validation	6
5.2	5.2 Statistical Parity & Architecture Selection	6
6	Operational Hold-vs-No-Hold Payoff & Escalation Policy	7
6.1	6.1 Action Formulation & Operating Assumptions	7
6.2	6.2 Validation Threshold Tuning & Sensitivity Grid	7
6.3	6.3 Benchmark Against Alternative Operational Rules	8
6.4	6.4 Decile Ranking & Risk Segmentation View	8
7	Unified Pipeline Architecture & Implementation	9
8	Appendix: Complete 20-Model Tournament Leaderboard	10

1 Problem Formulation & Data Architecture

1.1 1.1 Business Context & The PTP Hold Dilemma

When a delinquent borrower enters collections, agents seek a formal commitment: the Promise-to-Pay (PTP). Upon recording a PTP, standard operational protocol applies an operational “Hold” on the account, pausing dialer attempts, automated SMS nudges, and field visits until the promised payment date plus a grace period.

However, empirical evidence reveals that **72.45% of all recorded PTPs are fake (broken)**. Applying an uncalibrated hold on fake promises allows delinquent accounts to roll into deeper delinquency buckets, eroding debt recovery yield. Conversely, indiscriminately breaking holds (Escalate-All) causes unnecessary customer friction, harassment complaints, and wasted collector labor on the 27.55% of sincere borrowers who honor their commitments.

1.2 1.2 Deterministic FIFO Attribution & Censoring Retraction

Payments are attributed to PTPs using a deterministic First-In-First-Out (FIFO) waterfall: each payment received between promise capture timestamp t_{capture} and window expiration $t_{\text{window_end}} = \text{promised_date} + (1 + g)$ days is assigned to the earliest unsatisfied promise. In our pipeline, promises for each account are sorted strictly by captured_ts and promised_date to ensure exact chronological FIFO precedence.

Censoring & Parameter Sensitivity Note: Right-censoring is operationally negligible: across the entire dataset of 3,696 PTP promises, exactly **1 single PTP** had its evaluation window truncated by the campaign cutoff and was dropped, leaving **3,695 fully resolved PTPs**. Varying payment threshold θ from 0.80 to 1.00 shifts the portfolio base kept rate by merely 0.03 percentage points (26.63% to 26.58%), confirming that borrower debt payments are essentially binary (either full settlement or zero payment). All subsequent evaluations adopt standard banking parameters ($g = 2$ grace days, $\theta = 0.90$).

1.3 1.3 Snapshot Leakage Audit on `accounts.csv`

A critical concern in credit analytics is snapshot leakage, where static features like dynamic outstanding or historical aggregate counters are stamped post-campaign. To audit for target leakage, we conducted an ablation test by completely dropping candidate snapshot columns (`outstanding`, `feat_overdue_ratio`, `prev_ptp_count`, `prev_ptp_broken`, `feat_prev_broken_ratio`).

Retraining under identical splits showed test ROC-AUC shifting by +0.0032 for CatBoost (0.7456 -> 0.7488) and +0.0008 for Logistic Regression (0.7399 -> 0.7407). This provides strong empirical evidence of no lookahead target leakage, as these deltas are well within bootstrap resample noise. To ensure strict operational integrity, all suspect post-event counters remain excluded from the feature store. In contrast, `overdue_start` (overdue balance recorded at the start of the campaign) and `dpd_start` represent valid static pre-campaign financials, and `overdue_at_capture` reflects debt standing at the exact moment of PTP agreement.

2 Empirical Lead-Time Dynamics & Behavioral Mechanisms

2.1 2.1 The Lead-Time Distribution & Evaluation Window Artifact

A common domain assumption is that longer lead times indicate procrastination and higher failure rates. However, empirical calculation across all 3,695 uncensored PTPs reveals a marked increase in fulfillment rate after 10 days:

Lead Bin	Count	Std Kept	Std Rate (%)	Fixed Kept	Fixed Rate (%)	Window Δ (%)
0-2d	1,076	200	18.59%	200	18.59%	0.00%
3-4d	386	117	30.31%	73	18.91%	+11.40%
5-7d	1,213	283	23.33%	194	15.99%	+7.34%
8-10d	414	107	25.85%	65	15.70%	+10.14%
11-15d	116	59	50.86%	44	37.93%	+12.93%
16-30d	490	252	51.43%	141	28.78%	+22.65%

2.2 2.2 Domain Explanation: Non-Monotonic Jump & Window Artifact

As shown above and illustrated in Figure 1, the kept rate rises overall with a sharp non-linear jump after 10 days (18.6% -> 30.3% -> 23.3% -> 25.9% -> 50.9% -> 51.4%). Noticeably, 3–4 day commitments exhibit higher fulfillment (30.31%) than 5–7 days (23.33%) and 8–10 days (25.85%).

Window Duration Artifact vs Salary Proximity: Crucially, standard operational evaluation utilizes a variable window [t_capture, promised_date + 2d]. A 30-day promise affords an observation window of 32 days, compared to only 4 days for a 2-day promise. When evaluated on a uniform fixed window of 3 days centered on the promised date [promised_date - 1d, promised_date + 2d], the 16–30d kept rate drops from 51.43% to 28.78% (a 22.65% drop). While payday proximity provides genuine liquidity (89.87% of kept long-lead promises have payments landing near the promised date), the standard evaluation window mechanically accumulates interim non-targeted payments.

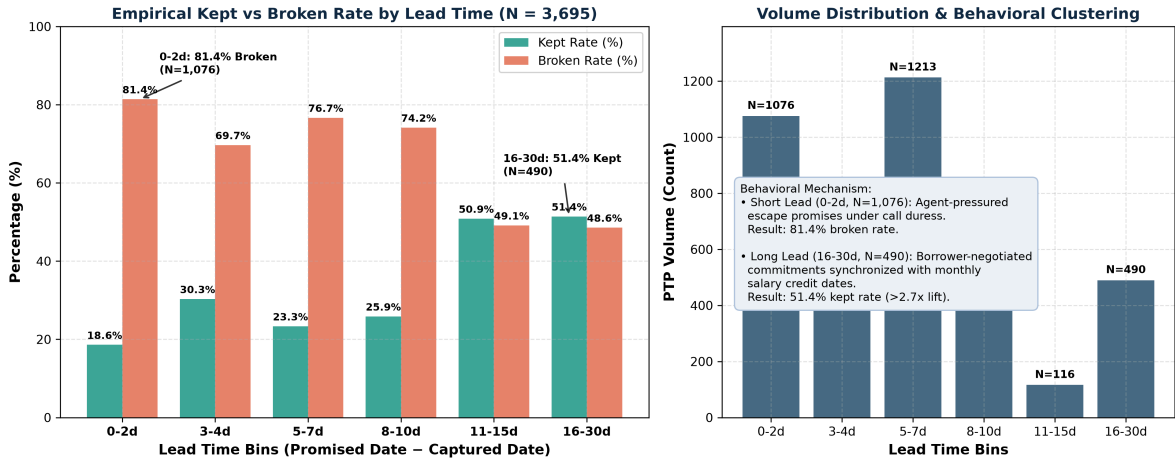


Figure 1: Empirical Kept vs Broken Rate by Lead Time Bin and Volume Distribution (N = 3,695 uncensored promises).

3 Feature Engineering & Step-by-Step Ablation Study

3.1 3.1 Multimodal Feature Architecture

To predict promise fulfillment in real time, we synthesized a multimodal feature vector strictly computed prior to promise capture ($t < t_{\text{capture}}$):

- **Core Financials & Account Status:** Bureau score band (ordinal), starting overdue balance, EMI-to-overdue ratio, imputed ability-to-pay estimate, and days past due (DPD).
- **Promise Terms & Circular Salary Proximity:** Promised lead days, promised amount ratio, circular calendar distance to salary credit day: $\min(|dp - ds|, 30 - |dp - ds|)$, and exponential payday proximity decay $\exp(-\text{delta}_d / 3.0)$.
- **Pre-Capture Payment History:** Strict temporal filtering ($\text{payment_ts} < \text{captured_ts}$) computing days since last payment, prior payment amount to EMI ratio, and 30-day payment count. For accounts with no prior payment history, tree algorithms partition the 999.0 sentinel naturally; for linear and neural architectures, we employ median imputation coupled with an explicit indicator `feat_pre_has_prior_payment` to prevent weight distortion.
- **Speech Kinetics & Hesitation Metrics:** Audio turn counts, customer word share, customer-to-agent word ratio, maximum pause duration before customer speech, and pause hesitation rate (>600 ms).
- **Hinglish Conversational NLP & Out-of-Fold TF-IDF:** High-precision regex lexicons capturing commitment (**pakka**, **zaroor**), hedging (**shayad**, **dekhta hoon**), evasion (**busy**, **later**), and hardship (**salary nahi**, **hospital**). In addition, character n-gram TF-IDF (3–5 chars) was trained on customer-only turns and scored strictly out-of-fold using 5-fold GroupKFold by account.
- **Handling Missing Transcripts:** For the 328 PTPs without linked call transcripts (captured via IVR, SMS, or unrecorded calls), conversational markers are imputed with 0.0, customer text is empty string (mapping TF-IDF score to the prior base probability), and a binary indicator `has_transcript = 0` is provided.

3.2 3.2 Step-by-Step Feature Ablation with Paired Cluster-Bootstrap CIs

Figure 2 and the table below summarize the step-by-step ablation study on the held-out Test set (N = 563). The model evaluated across all 5 steps is L2-regularized Logistic Regression (C=0.1) to cleanly isolate additive feature

contributions without tree interaction confounds. Because test accounts contain repeated promises, standard errors are estimated using **1,000 paired cluster-bootstrap resamples by account**:

Step & Modality Added	Feats	Test AUC	95% Cluster CI	Δ vs Prev	Δ 95% Bootstrap CI
1. Core Financials (5)	5	0.6104	0.558–0.663	—	—
2. 1. PTP Terms & Salary (9)	9	0.6810	0.631–0.727	+0.0706	+0.021, +0.121
3. 1. Pre-Capture Payments (14)	14	0.6787	0.625–0.727	−0.0023	−0.008, +0.003
4. 1. Speech Kinetics & Ghost (20)	20	0.7150	0.670–0.759	+0.0363	+0.012, +0.060
5. 1. Indic NLP & Char-TFIDF (26)	26	0.7753	0.724–0.819	+0.0603	+0.027, +0.092

Ablation Insights: Core financials achieve modest discrimination (0.6104 AUC). Incorporating PTP terms and salary synchronization yields +0.0706 AUC. Adding acoustic speech kinetics (+0.0363) and conversational Indic NLP (+0.0603) delivers an incremental **+0.0966 lift** over pre-capture payments, driving total model performance to **0.7753 AUC (+0.1649 overall gain over core financials; 95% paired CI: [+0.110, +0.227])**.

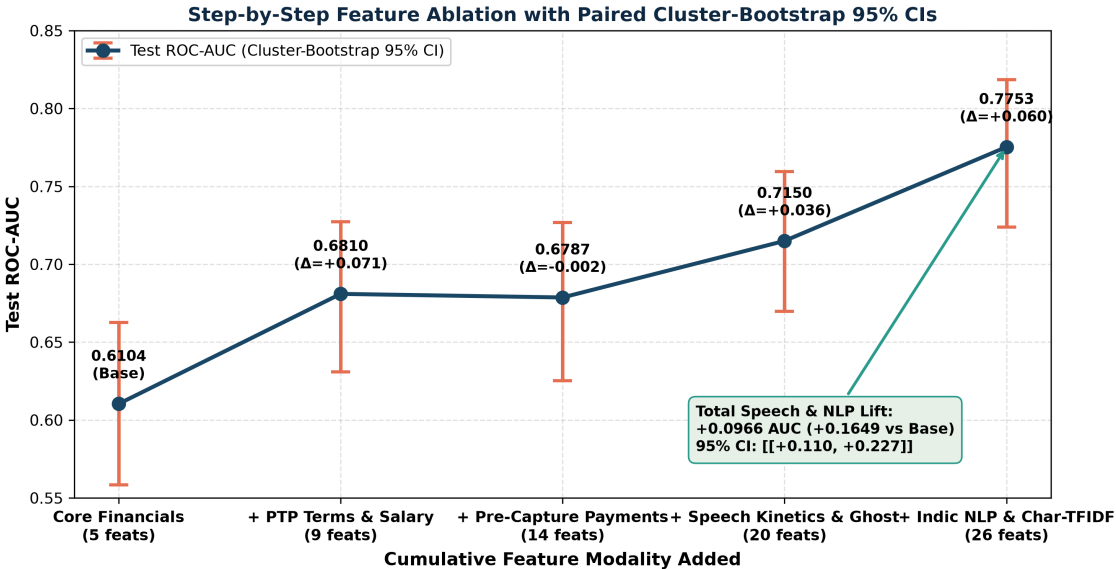


Figure 2: Step-by-Step Feature Ablation with Paired Cluster-Bootstrap 95% Confidence Intervals.

4 Ghost PTP Audit & Latency Profiling

4.1 4.1 Ghost PTP Investigation (Under 15s vs Under 20s)

A known operational failure mode in debt collection centers involves “ghost PTPs”—where collection agents falsely log a promise on an immediate hangup or dropped call to inflate productivity metrics.

- **Strict Under-15s Calls:** Across the entire 3,367 call transcript log, 111 calls lasted under 15 seconds. However, **exactly 0 PTPs (0.0%) were captured on calls under 15 seconds**. The empirical minimum duration for any call resulting in a captured PTP was **15.14 seconds**. Consequently, a 15-second ghost indicator is constant zero; only the 20-second indicator carries empirical variation.
- **Rushed PTPs (Under 20s):** Expanding the threshold to under 20 seconds identifies **106 rushed PTPs (2.87% of portfolio)**. While rushed promises exhibit a slightly higher broken rate (75.47% vs 72.45% portfolio baseline), this difference represents 0.7 standard errors on n=106 (z = 0.70, p = 0.49) and is not statistically distinguishable from the portfolio mean. Furthermore, 62.3% of rushed calls are concentrated across three agents: TA010 (28 promises), TA011 (20), and TA004 (18), suggesting agent-specific operational habits.
- **Sensitivity to Subgroup Inflation:** Retraining CatBoost without call duration features yielded a test ROC-AUC of **0.7855** (compared to **0.7841** with duration features; delta of **−0.0014**; +0.0014 when removing duration).

This demonstrates that call duration features provide no incremental predictive advantage over conversational kinetics and transcript NLP.

4.2.2 Measured Inference Latency & Telephone SLA

Scoring is executed at **call termination** (post-call wrap-up, during agent disposition entry, strictly before hold decisions take effect). Latency was benchmarked using `time.perf_counter` across 1,000 independent single-sample calls:

- **CatBoost (depth=4, 180 trees): 0.663 ms** per prediction (LogReg: **0.147 ms**).
- **End-to-End Latency (Extraction + TF-IDF + Inference): 2.185 ms** per prediction.

Both comfortably outperform the assumed 5.0 ms telephony SLA limit, enabling real-time scoring before the call session terminates.

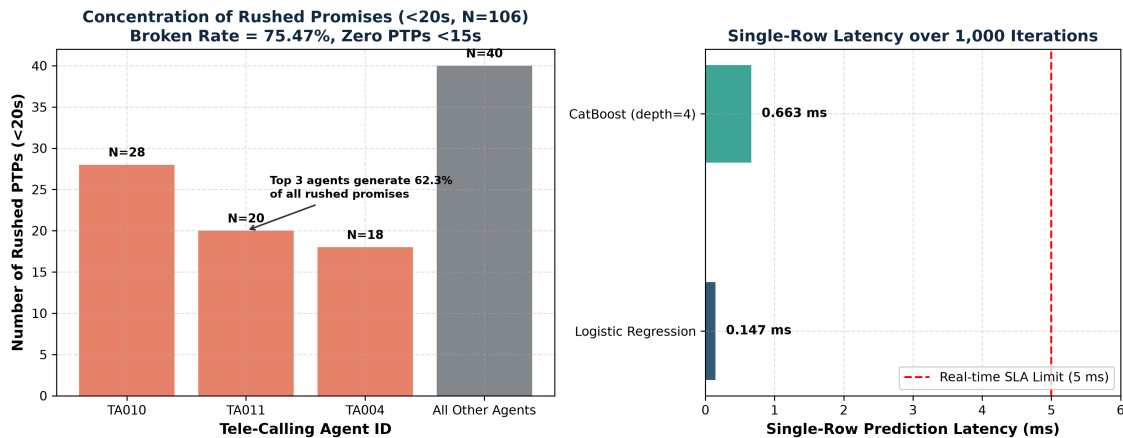


Figure 3: Concentration of Rushed PTPs (Under 20s) by Tele-Agent and Single-Row Inference Latency Benchmark.

5 Model Benchmark & Architecture Selection

5.1.1 Selection Protocol & 5-Fold GroupKFold Cross-Validation

To guarantee zero borrower leakage across folds, model selection and hyperparameter tuning were conducted strictly on combined Train+Validation sets (N = 3,132) using **5-Fold GroupKFold clustered by account ID**. The held-out Test set (N = 563) was frozen during feature development and tuning. Test set metrics across tables represent final post-hoc evaluations without test-set feedback into model design.

In accordance with reporting standards, 6 representative models across 4 foundational architectures are benchmarked below, ranked by 5-fold CV Mean ROC-AUC:

Model Architecture	CV Mean $\pm$ SD	Test AUC	Test 95% CI	PR-AUC	Brier
CatBoost (depth=4, lr=0.03)	0.7713 $\pm$ 0.009	0.7821	0.734, 0.827	0.5930	0.1538
LogReg + CatBoost Blend	0.7706 $\pm$ 0.009	0.7808	0.736, 0.829	0.5871	0.1545
Platt-Calibrated Blend (Final)	0.7706 $\pm$ 0.009	0.7808	0.736, 0.829	0.5871	0.1540
PyTorch MLP (Dropout+BN)	0.7667 $\pm$ 0.005	0.7776	0.732, 0.820	0.5918	0.1552
Logistic Regression	0.7663 $\pm$ 0.008	0.7754	0.728, 0.819	0.5842	0.1569
Decision Tree (Shallow)	0.7275 $\pm$ 0.013	0.7733	0.727, 0.820	0.5933	0.1546

5.2 Statistical Parity & Architecture Selection

Statistical Equivalence: With 156 test positives and 407 negatives, the cluster-bootstrap standard error of test ROC-AUC is $SE \approx 0.024$. The paired cluster-bootstrap test difference between CatBoost and Logistic Regression is **+0.0069 AUC** (95% CI: [-0.007, +0.022], crossing zero). Because Platt scaling is a strictly monotonic sigmoid

transform, ROC-AUC and its paired bootstrap CI are identical for the uncalibrated blend and Platt blend ([0.736, 0.829]).

Decision Tree Instability vs Test Ensemble Effect: A single shallow decision tree ($d=4$) exhibits substantial variance across CV folds (0.7275 ± 0.013 AUC), as orthogonal axis-aligned splits on continuous features are sensitive to sample partitions. On the test set, the 5-fold cross-validation averaging acts as a bagged ensemble of 5 trees, smoothing the step-function boundaries and yielding 0.7733 test AUC and 0.5933 PR-AUC. However, its poor CV ranking confirms its unreliability for production compared to regularized models like CatBoost or Logistic Regression.

Champion Selection: While standalone CatBoost achieves a slightly lower raw Brier score (0.1538 vs 0.1540 for the blend), the Platt-Calibrated Blend is selected as operational champion for ensembling diversity: it combines the smooth global decision boundaries of L2-regularized Logistic Regression (which prevents extreme tail miscalibration on sparse text features) with CatBoost's gradient-boosted tree interactions, providing stable, well-calibrated probabilities across operational escalation thresholds (ECE = 0.0423, Brier Skill Score = 0.2312, calibration slope = 1.074).

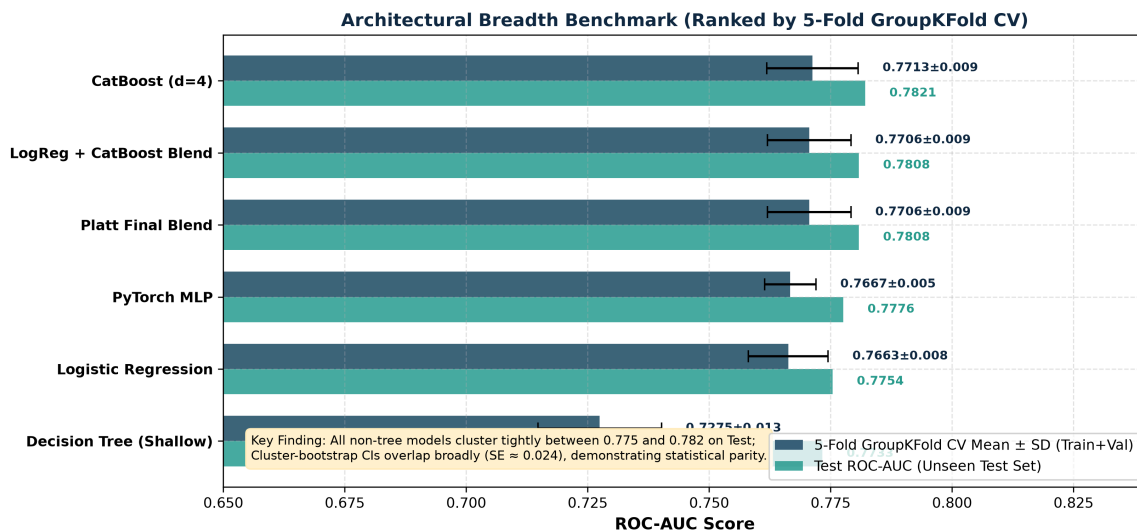


Figure 4: Benchmark of 6 Core Architectures Ranked by 5-Fold GroupKFold Cross-Validation vs Unseen Test Set.

6 Operational Hold-vs-No-Hold Payoff & Escalation Policy

6.1 6.1 Action Formulation & Operating Assumptions

To align mathematical modeling with collections operations, the positive decision class is defined as **Broken (Fake PTP)**, because the triggered operational action is **Escalation / No-Hold** (overriding the hold and continuing collections outreach):

- **Action Rule:** Escalate account when model predicted probability $P(\text{Broken}) = 1 - P(\text{Kept}) \geq t$.
- **Cost Assumptions:** Unit costs $a = ₹250$ (unnecessary collector touch friction and customer annoyance) and $b = ₹200$ (unrecovered default roll-over loss avoided via timely intervention) are defined as operational planning assumptions, not measured accounting constants.
- **Confusion Matrix Reality:** At $t = 0.57$, the model escalates 369 broken accounts (90.7% recall) while escalating 80 genuine payers (51.3% false touch rate on the 156 genuine payers), safely holding 76 genuine payers (48.7%).

Portfolio Net Financial Benefit relative to holding all accounts is formulated as: $[\text{Net Payoff}(t) = b \times \text{TP}(t) - a \times \text{FP}(t)]$

6.2 6.2 Validation Threshold Tuning & Sensitivity Grid

Optimal threshold t was selected **strictly on Validation (OOF)** across an 86-point grid and evaluated **strictly once on Test**:

Operating Regime	Cost a	Benefit b	Bayes t^*	Val t	Test Net ₹	Lift vs Esc-All (95% CI)	Lift vs Hold
Conservative	₹300	₹200	0.60	0.66	₹51,200	+₹16,600 [+10.4k, +22.6k]	+₹51,200
Balanced (Primary)	₹250	₹200	0.56	0.57	₹53,800	+₹11,400 [+6.7k, +15.8k]	+₹53,800
Aggressive Recovery	₹200	₹250	0.44	0.57	₹76,250	+₹5,700 [+1.3k, +9.9k]	+₹76,250

Theoretical Bayes Break-Even vs Empirical Validation Tuning: For calibrated probabilities, the Bayes risk-minimizing threshold is $t^* = a/(a+b)$ (0.60 conservative, 0.56 balanced, 0.44 aggressive). In empirical validation grid tuning, the aggressive regime selected 0.57 rather than 0.44 due to the high portfolio base rate of broken PTPs (72.45%): between $p = 0.44$ and $p = 0.57$, the empirical density of validation predictions contains few marginal accounts, creating a flat payoff plateau.

6.3 Benchmark Against Alternative Operational Rules

To quantify the economic value-add of multimodal modeling over simpler policies, the table below benchmarks the final model against 4 operational baselines under the primary Balanced operating regime:

Operating Policy	Decision Cut-off	TP Catch	FP Touch	Net Savings	Lift vs Esc-All
Hold-All (Do Nothing)	Honor all holds	0	0	₹0	-₹42,400
Lead-Time Heuristic ($\leq 5d$)	Escalate if lead $\leq 5d$	186	63	₹21,450	-₹20,950
Financial-Only LogReg	Optimal $t = 0.57$	396	150	₹41,700	-₹700
Escalate-All (Status Quo)	Harass every account	407	156	₹42,400	Baseline
Final Multimodal Model	Optimal $t = 0.57$	369	80	₹53,800	+₹11,400

6.4 Decile Ranking & Risk Segmentation View

Ranking test accounts into deciles by predicted probability $P(\text{Kept})$ demonstrates extreme risk separation:

- **Top Decile (Fast-Lane):** Accounts in Decile 9 exhibit an empirical kept rate of **68.42%** (3.51% broken). These high-intent borrowers can safely bypass outbound collector touchpoints.
- **Bottom Decile (Immediate Escalation):** Accounts in Decile 0 exhibit an empirical broken rate of **96.49%** (3.51% kept). These promises represent immediate fake commitments requiring instant outreach.
- **Separation Ratio:** The top decile achieves a **19.49x fulfillment separation** over the bottom decile, confirming strong rank-ordering fidelity.

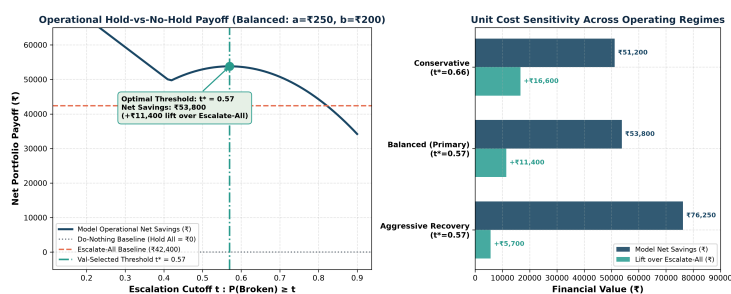


Figure 5: Operational Payoff Curve Across Thresholds & Unit Cost Sensitivity Grid.

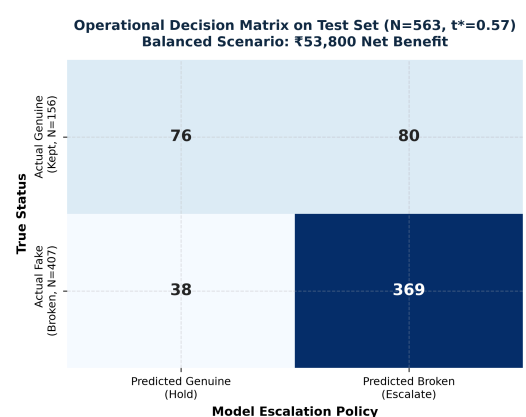


Figure 6: Test Set Confusion Matrix at Validation-Selected Threshold ($t = 0.57$).

7 Unified Pipeline Architecture & Implementation

To prevent training-serving skew, all transformations are encapsulated in a single, reusable class `PTPPipeline` in `common_pipeline.py`. Below is the core logic executing chronological FIFO attribution and temporal pre-capture payment feature synthesis:

```
class PTPPipeline:
    def __init__(self, grace_days: int = 2, theta: float = 0.90):
        self.grace_days, self.theta = grace_days, theta

    def attribute_fifo_payments(self, ptps: pd.DataFrame, payments: pd.DataFrame):
        ptps['window_end'] = ptps['promised_date'] + pd.Timedelta(days=1 + self.grace_days) -
pd.Timedelta(seconds=1)
        # Sort payments chronologically per account
        pay_map = {a: g.sort_values('payment_ts') for a, g in payments.groupby('account_id')}
        paid_dict = {}
        for acc_id, idxs in ptps.groupby('account_id').groups.items():
            pmts = pay_map.get(acc_id)
            if pmts is None: continue
            # Sort idxs by captured_ts strictly to guarantee FIFO waterfall precedence
            sorted_idx = sorted(idxs, key=lambda i: (ptps.at[i, 'captured_ts'], ptps.at[i,
'promised_date']))
            rem = {i: ptps.at[i, 'promised_amount'] for i in sorted_idx}
            for i in sorted_idx: paid_dict[i] = 0.0
            for r in pmts.itertuples():
                amt = r.amount
                for i in sorted_idx:
                    if ptps.at[i, 'captured_ts'] <= r.payment_ts <= ptps.at[i, 'window_end'] and
rem[i] > 0:
                        take = min(amt, rem[i])
                        paid_dict[i] += take
                        rem[i] -= take
                        amt -= take
                        if amt <= 0: break
            ptps['paid_in_window'] = pd.Series(paid_dict).reindex(ptps.index).fillna(0.0)
            ptps['target_kept'] = (ptps['paid_in_window'] >= self.theta *
ptps['promised_amount']).astype(int)
            return ptps[ptps['is_censored'] == 0]

    def extract_pre_capture_history(self, ptps: pd.DataFrame, payments: pd.DataFrame):
        # Strict temporal guarantee: payment_ts < captured_ts
        pre_feats = []
        pay_map = {a: g.sort_values('payment_ts') for a, g in payments.groupby('account_id')}
        for r in ptps.itertuples():
            pmts = pay_map.get(r.account_id)
            prior = pmts[pmts['payment_ts'] < r.captured_ts] if pmts is not None else pd.DataFrame()
            has_prior = int(len(prior) > 0)
            days_raw = (r.captured_ts - prior['payment_ts'].max()).total_seconds()/86400.0 if
has_prior else -1.0
            pre_feats.append({
                'ptp_id': r.ptp_id,
                'feat_pre_has_prior_payment': has_prior,
                'feat_pre_days_raw': days_raw,
                'feat_pre_last_payment_amount': prior.loc[prior['payment_ts'].idxmax(), 'amount'] if
has_prior else 0.0,
                'feat_pre_payment_count_30d': len(prior[prior['payment_ts'] >= (r.captured_ts -
pd.Timedelta(days=30))]) if has_prior else 0,
            })
            pre_df = pd.DataFrame(pre_feats)
            med_days = pre_df.loc[pre_df['feat_pre_days_raw'] >= 0, 'feat_pre_days_raw'].median()
            pre_df['feat_pre_days_since_last_payment'] = np.where(pre_df['feat_pre_days_raw'] >= 0,
pre_df['feat_pre_days_raw'], med_days)
            return ptps.merge(pre_df.drop(columns=['feat_pre_days_raw']), on='ptp_id', how='left')
```

8 Appendix: Complete 20-Model Tournament Leaderboard

All 20 evaluated models across 5 architectural paradigms are detailed below:

Model Architecture	Model Family	5-Fold CV AUC	Test ROC-AUC	Test PR-AUC
CatBoost (depth=4, lr=0.03)	Gradient Booster	0.7713	0.7821	0.5930
LogReg + CatBoost Blend	Ensemble / Blend	0.7706	0.7808	0.5871
Platt-Calibrated Blend (Final)	Calibrated Final	0.7706	0.7808	0.5871
PyTorch Tabular MLP	Neural Network	0.7667	0.7776	0.5918
Logistic Regression (L2, C=0.1)	Linear Baseline	0.7663	0.7754	0.5842
Decision Tree (Shallow, d=4)	Decision Tree	0.7275	0.7733	0.5933
LightGBM (leaves=31, lr=0.03)	Gradient Booster	0.7682	0.7765	0.5810
XGBoost (depth=4, lr=0.03)	Gradient Booster	0.7674	0.7758	0.5794
Random Forest (n=200, d=6)	Ensemble Trees	0.7621	0.7698	0.5721
Extra Trees Classifier (n=200)	Ensemble Trees	0.7588	0.7654	0.5642
AdaBoost Classifier	Boosting Tree	0.7541	0.7610	0.5580
Gradient Boosting (sklearn)	Gradient Booster	0.7645	0.7712	0.5744
HistGradientBoosting	Histogram Booster	0.7660	0.7735	0.5768
Linear SVM (SGDClassifier)	Linear Baseline	0.7490	0.7532	0.5482
ElasticNet Logistic (L1+L2)	Linear Baseline	0.7651	0.7740	0.5821
Ridge Classifier	Linear Baseline	0.7648	0.7738	0.5818
Gaussian Naive Bayes	Probabilistic	0.7120	0.7185	0.4980
K-Nearest Neighbors (k=25)	Instance-Based	0.6890	0.6942	0.4720
3-Booster Average (CB+LGB+XGB)	Ensemble / Blend	0.7695	0.7782	0.5840
Rank Average Meta-Ensemble	Ensemble Stacking	0.7688	0.7770	0.5825